

MyExperience

- We ask for a lot of feedback in this course
- This is the most important feedback you will provide

MyExperience

- COMP1511/COMP1911 requires a lot of resources to deliver
- If you want more courses like this one, please complete the form
- Completely anonymous, can not be identified
- Run by UNSW, not School or Faculty

MyExperience Completion Tiers

>70%
Bonus Jake + Henry +
tutors Q&A
chill/livestream/games

> 80%
??? (we've never hit it)

Complete now



COMP1911 Computing
1A



COMP1511
Programming
Fundamentals

Linked Lists recap

Animated explainer:

[https://www.youtube.com/watch?
v=CTW_UOz9d6Q](https://www.youtube.com/watch?v=CTW_UOz9d6Q)

LL Code Review

Today

- Revision
- Exam-style QNs
- Demos
- Kahoot

Spotify Recap

Freeing Memory:

`delete_spotify`

The Strategy: Deconstruct from the Bottom-Up

Inner loop for songs

```
while (current_song != NULL) {  
    // 1. Store a pointer to the  
    next song. If we don't,  
    //     freeing current_song  
    makes current_song->next invalid.  
    next_song = current_song-  
    >next;  
  
    // 2. Free the current node.  
    free(current_song);  
  
    // 3. Move to the next node  
    using our saved pointer.  
    current_song = next_song;  
}
```

Outer loop for playlists

```
while (current_playlist != NULL) {  
    // The inner loop runs here,  
    freeing all songs...  
  
    // 1. Once the songs are gone, we  
    save the next playlist.  
    next_playlist = current_playlist-  
    >next;  
  
    // 2. Free the now-empty current  
    playlist.  
    free(current_playlist);  
  
    // 3. Move to the next playlist to  
    continue the process.  
    current_playlist = next_playlist;  
}
```

Implementation: Nested Loops

The "bottom-up" strategy translates directly into nested loops. An outer loop iterates through the playlists, and an inner loop iterates through the songs of each playlist.

```
void delete_spotify(struct spotify *spotify) {
    struct playlist *current_playlist = spotify->playlists;
    struct playlist *next_playlist;

    // Outer loop for playlists
    while (current_playlist != NULL) {
        struct song *current_song = current_playlist->songs;
        struct song *next_song;

        // Inner loop for songs
        while (current_song != NULL) {
            next_song = current_song->next;
            free(current_song);
            current_song = next_song;
        }

        next_playlist = current_playlist->next;
        free(current_playlist);
        current_playlist = next_playlist;
    }
    free(spotify);
}
```

Exam-style linked list questions

```
#include <stdio.h>
#include <assert.h>
#include <stdlib.h>

struct node {
    struct node *next;
    int data;
};

// delete_divSix should delete the first node that is divisible by 6
// If there are no nodes that are divisible by 6, it should return
// the list unchanged.
struct node *delete_div_six(struct node *head) {
    return NULL;
}

////////////////////////////////////
// DO NOT CHANGE THIS CODE BELOW
////////////////////////////////////
void print_list(struct node *head) {
    struct node *current = head;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
}

// DO NOT CHANGE THIS MAIN FUNCTION
int main(int argc, char *argv[]) {
    // create linked list from command line arguments
    struct node *head = strings_to_list(argc - 1, argv[1]);

    // If you're getting an error here,
    // you have returned an uninitialized value
    struct node *new_head = delete_div_six(head);
    print_list(new_head);

    return 0;
}

// DO NOT CHANGE THIS FUNCTION
// create linked list from array of strings
struct node *strings_to_list(int len, char *strings[]) {
    struct node *head = NULL;
    for (int i = len - 1; i >= 0; i--) {
        struct node *n = malloc(sizeof(struct node));
        assert(n != NULL);
        n->next = head;
        n->data = atoi(strings[i]);
        head = n;
    }
    return head;
}

// print linked list
void print_list(struct node *head) {
    printf("\n");
    for (struct node *n = head; n != NULL; n = n->next) {
        // If you're getting an error here,
        // you have returned an invalid list
        printf("%d ", n->data);
    }
}
```



COMP1911 Computing
1A



COMP1511
Programming
Fundamentals